

SBA301

BUỔI 05 / SESSION 05

INTEGRATED REACT LAB 01

Orchid Gallery with Components, Props, State & React-Bootstrap

Loại buổi: Integrated Lab / Consolidation Workshop • Thời lượng: 135 phút

Mục tiêu: tích hợp và chứng minh năng lực React nền tảng trước khi học React Router.

UNDERSTAND	DESIGN	IMPLEMENT	RUN	VERIFY	DEBUG	COMMIT	EXPLAIN
Hiểu yêu cầu	Vẽ cây component	Làm từng bước nhỏ	Chạy ngay	Đối chiếu kết quả	Sửa theo evidence	Lưu checkpoint	Giải thích không AI

Quy tắc của buổi lab — Không chép một solution hoàn chỉnh rồi chạy một lần. Mỗi thay đổi quan trọng phải được chạy, kiểm tra và giải thích trước khi chuyển bước.

1. Tổng quan buổi học

Buổi 05 là integration checkpoint đầu tiên của phần React. Thay vì mở thêm một chapter lớn, sinh viên ghép kiến thức đã học ở Buổi 02–04 thành một ứng dụng frontend chạy được: Vite → Component Architecture → React-Bootstrap → Orchid Data → Props → State → Event → Detail/Modal → Verification → Git.

1.1. Kết nối chuỗi học tập

Buổi	Năng lực chính	Trạng thái sau buổi
02	Tạo và chạy React app	I can create React app.
03	Tổ chức UI bằng components	I can structure React app with components.
04	Props, State, Hooks, Context	I understand how data flows and changes.
05	Tích hợp và kiểm chứng	I can combine everything into one working application.
06	React Router & Navigation	I can navigate among SPA views.

1.2. Sản phẩm trung tâm

Sinh viên xây một Orchid Gallery gồm NavBar, danh sách Orchid dạng Card, ảnh, thông tin cơ bản, và một tương tác View Detail điều khiển bởi state. Modal/detail được dùng như nội dung cầu nối từ Lab 02, không phải yêu cầu gốc của Lab 01.

ORCHID GALLERY

NavBar

↓

OrchidList

↓ map()

OrchidCard × N

↓ Props

Orchid data

View Detail → Event → State → Modal/Detail

2. Bản đồ nguồn học liệu và ranh giới nội dung

Buổi 05 phải phân biệt rõ nguồn để sinh viên hiểu đâu là Lab 01 chính thức, đâu là kiến thức chapter đã học, và đâu là bridge từ Lab 02.

Nhóm nội dung	Bao gồm	Cách sử dụng trong Buổi 05
Official Lab 01 Core	Environment, Vite, React app, React-Bootstrap, NavBar, Orchid list, Card, images, run project	Là trực thực hành chính.
Consolidated Chapter Knowledge	Component Architecture, Props, State, useState, Events, Hook rules	Dùng để nâng Lab 01 thành bài tích hợp có giải thích data flow.
Lab 02 Bridge	Structured Orchid data, Detail, Modal	Chỉ dùng như cầu nối để luyện state; phải ghi nhãn rõ.

Nguồn chính xác — Không ghi rằng Modal/Detail là yêu cầu cốt lõi của Lab 01. Structured Orchid data và Modal detail được xem là bridge từ Lab 02.

3. Mục tiêu học tập và bằng chứng

Năng lực	Sinh viên phải làm được	Bằng chứng
Run & diagnose	Chạy existing Vite project và chẩn đoán setup failure trước khi sửa source.	URL dev server + terminal không lỗi blocking.
Architecture	Tổ chức UI thành NavBar, OrchidList, OrchidCard, Footer và vẽ component tree.	Component tree + responsibility table.
React-Bootstrap	Cài/kiểm tra package, import CSS, dùng Grid/Card.	UI responsive, Bootstrap style hiển thị.
Data & Props	Tạo/reuse data module, truyền object prop vào OrchidCard.	Trace một record từ data → prop → JSX.
Collection rendering	Dùng map() và stable key để render nhiều Card.	Số Card khớp số record.
State & Event	Dùng useState và event-driven interaction để mở/đóng detail.	Predict state trước click + interaction hoạt động.
Debugging	Dùng terminal, console và evidence để xác định first broken stage.	Bug log/evidence.
Git & explanation	Tạo commit checkpoint và giải thích app không AI.	git log -1 + oral explanation.

4. Pre-Lab Human Check (không dùng AI)

1. `App.jsx` có vai trò gì trong project hiện tại?
2. React Component là gì?
3. Component Tree dùng để mô tả điều gì?
4. React-Bootstrap là gì?
5. Prop là gì?
6. Child có được sửa trực tiếp Props đã nhận không?
7. State là gì?
8. `useState` trả về hai giá trị gì về mặt khái niệm?
9. Event handler làm gì?
10. Lỗi runtime/browser của React thường xem ở đâu?

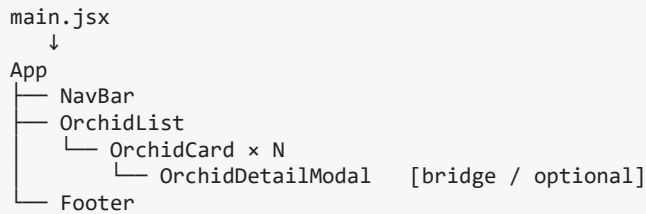
Ngưỡng sẵn sàng — Nếu đúng dưới 7/10 câu, dành 5 phút ôn đúng phần thiếu thay vì tiếp tục theo lab trong trạng thái không hiểu.

5. Tiêu chí hoàn thành trước khi bắt đầu code

- ☐ Application khởi động và browser mở đúng dev URL.
- ☐ NavBar hiển thị và không crash khi resize.
- ☐ Orchid list hiển thị nhiều Card.
- ☐ Ảnh Orchid load đúng từ public/images.
- ☐ Chỉ một OrchidCard component được tái sử dụng.
- ☐ Các Card khác nhau hiển thị dữ liệu khác nhau qua Props.
- ☐ map() được dùng để render collection và có stable key.
- ☐ Một tương tác dùng useState hoạt động.
- ☐ Detail/Modal mở và đóng nếu thực hiện bridge task.
- ☐ Terminal không có compile error blocking.
- ☐ Browser console không có relevant error.
- ☐ Có Git checkpoint.
- ☐ Sinh viên giải thích được data flow không dùng AI.

6. Kiến trúc ứng dụng trước khi code

Component tree mục tiêu



Component	Trách nhiệm	Dữ liệu/State chính
App	Compose toàn ứng dụng	Không cần ôm mọi logic.
NavBar	Navigation UI placeholder	Chưa dùng React Router.
OrchidList	Tổ chức collection, map(), grid	Orchid collection.
OrchidCard	Hiển thị một Orchid	Prop: orchid; local state nếu dùng Design A.
OrchidDetailModal	Hiển thị thông tin chi tiết	show / selected orchid tùy thiết kế.
Footer	Footer UI	Không có data phức tạp.

Thiết kế trước code — Mọi sinh viên phải vẽ cây component và nói được trách nhiệm của từng component trước khi mở AI hoặc viết component mới.

7. Quy trình thực hành zero-assumption

Mỗi bước có 5 phần: **THAO TÁC** → **VÌ SAO** → **KẾT QUẢ MONG ĐỢI** → **KIỂM TRA** → **NẾU THẤT BẠI**. Không chuyển bước khi checkpoint của bước hiện tại chưa đạt.

Bước 0. Mở đúng project và chạy baseline

THAO TÁC: Trong IntelliJ IDEA mở project đã dùng ở Buổi 04. Mở Terminal tại project root và chạy `npm run dev`.

VÌ SAO: Phải chứng minh project cũ đang hoạt động trước khi thêm lỗi mới.

KẾT QUẢ MONG ĐỢI: Terminal in Local URL của Vite và browser tải được ứng dụng.

KIỂM TRA: Kiểm tra có `src/``, `public/``, `package.json``; URL mở được; console không có lỗi blocking.

NẾU THẤT BẠI: Không sửa source. Đọc first terminal error; kiểm tra working directory và dependencies trước.

```
npm run dev
```

Bước 1. Kiểm tra working directory

THAO TÁC: Ở PowerShell chạy `Get-Location`` hoặc đọc terminal prompt. Xác nhận thư mục hiện tại chứa `package.json``.

VÌ SAO: Rất nhiều lỗi npm của người mới chỉ là chạy lệnh sai thư mục.

KẾT QUẢ MONG ĐỢI: Project root chính xác.

KIỂM TRA: `package.json`` hiện trong Project tool window và terminal đang ở cùng root.

NẾU THẤT BẠI: Dùng ``cd`` vào đúng project root rồi chạy lại lệnh.

```
Get-Location
# sau đó kiểm tra package.json
```

Bước 2. Kiểm tra dependency

THAO TÁC: Mở ``package.json``, xác nhận có ``react``, ``react-dom``, ``react-bootstrap``, ``bootstrap``. Chỉ cài bổ sung nếu thiếu.

VÌ SAO: Buổi lab ưu tiên VERIFY hơn REINSTALL.

KẾT QUẢ MONG ĐỢI: Không có unresolved package khi chạy.

KIỂM TRA: Nếu thiếu React-Bootstrap, chạy command bên dưới và kiểm tra ``package.json`` sau khi hoàn tất.

NẾU THẤT BẠI: Nếu ``npm install`` báo lỗi, lưu exact error và không xóa project để làm lại ngay.

```
npm install react-bootstrap bootstrap
```

Bước 3. Kiểm tra Bootstrap CSS import

THAO TÁC: Trong entry file phù hợp (``main.jsx`` theo project hiện tại), xác nhận có import Bootstrap CSS.

VÌ SAO: Package được cài không đồng nghĩa stylesheet được load.

KẾT QUẢ MONG ĐỢI: Component React-Bootstrap có Bootstrap styling.

KIỂM TRA: Resize/browser refresh; so sánh style trước/sau.

NẾU THẤT BẠI: Kiểm tra đúng đường import và dependency ``bootstrap`` đã tồn tại.

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Bước 4. Vẽ component tree và bảng trách nhiệm

THAO TÁC: Vẽ cây ``App`` → ``NavBar`` / ``OrchidList`` → ``OrchidCard`` / ``Footer``; nếu làm detail bridge thì thêm ``OrchidDetailModal``.

VÌ SAO: Thiết kế trách nhiệm và data flow trước khi viết code giảm việc dồn mọi thứ vào ``App.jsx``.

KẾT QUẢ MONG ĐỢI: Mỗi component có một trách nhiệm hiểu được.

KIỂM TRA: Một thành viên giải thích tree trong 60 giây; reviewer hỏi “data ở đâu?”, “state ở đâu?”.

NẾU THẤT BẠI: Nếu tree có nhiều wrapper vô nghĩa, gộp lại; nếu App chứa mọi thứ, tách phần có trách nhiệm rõ.

8. NavBar và assets

Bước 5. Tạo hoặc kiểm tra NavBar

THAO TÁC: Dùng React-Bootstrap ``Navbar``, ``Container``, ``Nav``, ``Navbar.Brand``, ``Navbar.Toggle``, ``Navbar.Collapse``, ``Nav.Link`` theo Lab 01.

VÌ SAO: Hoàn thiện phần UI navigation trước khi học Router.

KẾT QUẢ MONG ĐỢI: Brand và các link placeholder hiển thị; responsive toggle không crash.

KIỂM TRA: Resize browser và kiểm tra console.

NẾU THẤT BẠI: Nếu import error, kiểm tra package/import path. Không thêm React Router ở buổi này.

```
import { Container, Nav, Navbar } from "react-bootstrap";

function AppNavBar() {
  return (
    <Navbar bg="dark" data-bs-theme="dark" expand="md">
      <Container>
        <Navbar.Brand href="#">Orchid Gallery</Navbar.Brand>
        <Navbar.Toggle aria-controls="main-nav" />
        <Navbar.Collapse id="main-nav">
          <Nav className="ms-auto">
            <Nav.Link href="#home">Home</Nav.Link>
            <Nav.Link href="#orchids">Orchids</Nav.Link>
            <Nav.Link href="#about">About</Nav.Link>
          </Nav>
        </Navbar.Collapse>
      </Container>
    </Navbar>
  );
}

export default AppNavBar;
```

Nhận diện — `Nav.Link` ở đây chỉ là navigation UI placeholder. Nó chưa phải route của React Router.

Bước 6. Chuẩn bị thư mục ảnh

THAO TÁC: Đặt resource ảnh của course vào `public/images/` theo cấu trúc Lab 01.

VÌ SAO: Vite phục vụ tài nguyên trong `public` bằng đường dẫn từ root URL.

KẾT QUẢ MONG ĐỢI: Một ảnh có thể truy cập bằng `/images/<filename>`.

KIỂM TRA: Mở trực tiếp URL ảnh trong browser hoặc dùng Network tab.

NEU THẤT BẠI: Kiểm tra filename, chữ hoa/thường, phần mở rộng và đường dẫn data. Không thêm `public` vào URL browser.

```
public/
├── images/
│   ├── 4n.jpg
│   ├── mini-yaya.jpg
│   └── ...
```

Browser path example:
/images/4n.jpg

9. Orchid Data Module

Structured Orchid data được sử dụng như bridge từ Lab 02. Trong bài tích hợp này, data module chỉ là static frontend source data — KHÔNG phải database.

Bước 7. Tạo data module

THAO TÁC: Tạo `src/shared/OrchidsData.js` (hoặc giữ đúng filename của Lab 02 nếu đang tái hiện tài nguyên chính thức). Export một mảng dữ liệu.

VÌ SAO: Tách static data khỏi component giúp data flow dễ quan sát và chuẩn bị cho REST API sau này.

KẾT QUẢ MONG ĐỢI: File import được, app vẫn compile.

KIỂM TRA: Không có syntax error; mảng có id ổn định và image path đúng.

NẾU THẤT BẠI: Kiểm tra export/import trước khi sửa JSX.

```
export const OrchidsData = [
  {
    id: "1",
    orchidName: "Ceasar 4N",
    origin: "Taiwan",
    color: "Purple",
    category: "Dendrobium",
    isSpecial: true,
    rating: 4.8,
    image: "/images/4n.jpg"
  },
  {
    id: "2",
    orchidName: "Mini Yaya",
    origin: "Vietnam",
    color: "Yellow",
    category: "Dendrobium",
    isSpecial: false,
    rating: 4.5,
    image: "/images/mini-yaya.jpg"
  }
  // Bổ sung record theo course resource
];
```

Source boundary — Các field như origin, color, isSpecial, rating, category và detail được dùng như consolidation/bridge từ Lab 02. Chỉ dùng field thật sự có trong data resource của lớp.

10. Một OrchidCard chạy đúng trước khi render cả danh sách

Bước 8. Tạo `OrchidCard.jsx`

THAO TÁC: Tạo `src/components/OrchidCard.jsx`. Component chỉ nhận một prop `orchid` và render một record bằng React-Bootstrap Card.

VÌ SAO: Một reusable item component là trung tâm của component architecture và Props.

KẾT QUẢ MONG ĐỢI: Một Card hiển thị ảnh, tên, category và special indicator.

KIỂM TRA: Tạm render một record duy nhất từ parent. Console không lỗi.

NẾU THẤT BẠI: Nếu undefined, trace field name và prop name trước khi thay code hàng loạt.

```
import { Badge, Button, Card } from "react-bootstrap";

function OrchidCard({ orchid }) {
  return (
    <Card className="h-100 shadow-sm">
      <Card.Img
        variant="top"
        src={orchid.image}
        alt={orchid.orchidName}
      />
      <Card.Body className="d-flex flex-column">
        <Card.Title>{orchid.orchidName}</Card.Title>
        <Card.Text className="mb-2">
          Category: {orchid.category}
        </Card.Text>
        <div className="mb-3">
          {orchid.isSpecial && <Badge bg="warning">Special</Badge>}
        </div>
      </Card.Body>
    </Card>
  );
}
```

```

    </div>
    <Button className="mt-auto">View Detail</Button>
  </Card.Body>
</Card>
);
}

export default OrchidCard;

```

Bước 9. Truyền object qua Props và trace data

THAO TÁC: Ở parent/list component, import data và truyền `orchid={OrchidsData[0]}` vào một OrchidCard để test một record.

VÌ SAO: Làm một record chạy đúng trước khi render nhiều record giúp cô lập bug.

KẾT QUẢ MONG ĐỢI: Card hiển thị đúng record đầu.

KIỂM TRA: Điền trace: source object → prop name → receiving component → rendered field.

NẾU THẤT BẠI: Nếu card trống, kiểm tra index, import data, prop name và destructuring name theo đúng thứ tự.

```
<OrchidCard orchid={OrchidsData[0]} />
```

Trace	Giá trị sinh viên điền
Source object	_____
Prop name	_____
Receiver	_____
Rendered field	_____

11. Render collection bằng map() và key

Ba component hard-coded có thể chạy, nhưng không scale. Buổi 05 chuyển từ “nhiều JSX lặp lại” sang “array → map() → reusable component”.

Nhận diện quan trọng — `map()` là JavaScript Array method, không phải React Hook.

Bước 10. Tạo OrchidList và dùng map()

THAO TÁC: Tạo `OrchidList.jsx`, import `OrchidsData`, dùng `map()` để tạo một `OrchidCard` cho mỗi record.

VÌ SAO: Đây là cách nối collection data với reusable item component.

KẾT QUẢ MONG ĐỢI: Số OrchidCard render bằng số record của mảng.

KIỂM TRA: Đếm data length và card count; kiểm tra warning trong console.

NẾU THẤT BẠI: Nếu callback không tạo UI, kiểm tra return/parentheses; nếu field sai, trace một record cụ thể.

```

import { Container, Row, Col } from "react-bootstrap";
import { OrchidsData } from "../shared/OrchidsData";
import OrchidCard from "../OrchidCard";

function OrchidList() {
  return (

```



```

<Container id="orchids" className="py-4">
  <h2 className="mb-4">Orchid Collection</h2>
  <Row className="g-4">
    {OrchidsData.map((orchid) => (
      <Col key={orchid.id} xs={12} sm={6} lg={4}>
        <OrchidCard orchid={orchid} />
      </Col>
    ))}
  </Row>
</Container>
);
}

export default OrchidList;

```

`key={orchid.id}` là stable identifier giúp React theo dõi phần tử trong collection. `key` không phải normal prop để child đọc bằng `props.key`; nếu child cần id thì truyền object hoặc một prop id bình thường.

Bước 11. Kiểm tra responsive grid

THAO TÁC: Resize browser ở desktop, tablet và mobile width.

VÌ SAO: Container/Row/Col phải tổ chức được Card trên nhiều kích thước màn hình.

KẾT QUẢ MONG ĐỢI: Ví dụ cấu hình `xs=12`, `sm=6`, `lg=4`: mobile 1 cột, sm 2 cột, lg 3 cột.

KIỂM TRA: Quan sát cards có overflow hay không; ảnh và text có bị cắt bất thường không.

NEẾU THẤT BẠI: Nếu layout không đúng, kiểm tra nesting Container → Row → Col và Bootstrap CSS trước khi chỉnh CSS tùy ý.

Kiểm tra list	Expected	Actual	PASS/FAIL
Record count	Theo `OrchidsData.length`	_____	_____
Card count	Bằng record count	_____	_____
Images	Tất cả load	_____	_____
Names	Tất cả đúng	_____	_____
Console	Không relevant error	_____	_____

12. Interaction: useState → Event → Detail

Bridge content — Detail/Modal là nội dung cầu nối từ Lab 02, được đưa vào để củng cố State và event-driven UI. Không xem đây là core requirement nguyên bản của Lab 01.

12.1. Chọn thiết kế state

Thiết kế	State owner	Khi phù hợp
A — Local Modal State	Mỗi OrchidCard giữ `show`	Dễ hiểu, phù hợp lớp mới; mỗi Card tự mở detail của chính nó.
B — Parent Selection State	OrchidList giữ `selectedOrchid` + `showModal`	Nâng cao hơn; một Modal dùng chung cho toàn list.

Core path cho lớp mới: Design A. Chỉ yêu cầu một thiết kế, không làm cả hai nếu thời gian hạn chế.

Bước 12. Thêm local state vào OrchidCard

THAO TÁC: Import `useState` và khai báo `const [show, setShow] = useState(false);`.

VÌ SAO: Component cần nhớ trạng thái detail đang đóng hay mở.

KẾT QUẢ MONG ĐỢI: App compile; ban đầu detail không hiển thị.

KIỂM TRA: Sinh viên phải nói: `show` = current state, `setShow` = updater, `false` = initial value.

NẾU THẤT BẠI: Nếu Hook error, kiểm tra import và Rules of Hooks: gọi ở top-level của function component.

```
import { useState } from "react";

function OrchidCard({ orchid }) {
  const [show, setShow] = useState(false);
  // ...
}
```

Bước 13. Nối Button với event

THAO TÁC: Thay Button thành `onClick={() => setShow(true)}`.

VÌ SAO: Click phải tạo event → gọi state updater → state đổi → UI render lại.

KẾT QUẢ MONG ĐỢI: Trước click `show=false`; sau click `show=true`.

KIỂM TRA: Bắt buộc dự đoán state trước khi click rồi mới chạy.

NẾU THẤT BẠI: Không viết `onClick={setShow(true)}` vì biểu thức đó chạy ngay trong render.

```
<Button onClick={() => setShow(true)}>
  View Detail
</Button>
```

Bước 14. Thêm Modal/Detail UI

THAO TÁC: Dùng React-Bootstrap Modal nếu lớp thực hiện bridge task; binding `show={show}` và `onHide={() => setShow(false)}`.

VÌ SAO: Modal là biểu hiện trực quan của declarative state-driven UI.

KẾT QUẢ MONG ĐỢI: Click View Detail mở Modal; Close hoặc X đóng Modal.

KIỂM TRA: Kiểm tra Open và Close độc lập; console không lỗi.

NẾU THẤT BẠI: Nếu không đóng, trace `show prop` → onHide → setShow(false) theo đúng thứ tự.

```
import { Badge, Button, Card, Modal } from "react-bootstrap";

<Modal show={show} onHide={() => setShow(false)} centered>
  <Modal.Header closeButton>
    <Modal.Title>{orchid.orchidName}</Modal.Title>
  </Modal.Header>

  <Modal.Body>
    <img
      src={orchid.image}
      alt={orchid.orchidName}
      className="img-fluid rounded mb-3"
    />
    <p><strong>Category:</strong> {orchid.category}</p>
    {orchid.origin && <p><strong>Origin:</strong> {orchid.origin}</p>}
    {orchid.color && <p><strong>Color:</strong> {orchid.color}</p>}
    {orchid.rating !== undefined && (
      <p><strong>Rating:</strong> {orchid.rating}</p>
    )}
  </Modal.Body>

  <Modal.Footer>
    <Button variant="secondary" onClick={() => setShow(false)}>
      Close
    </Button>
  </Modal.Footer>
</Modal>
```

12.2. Props vs State checkpoint

Giá trị	Loại	Vì sao
`orchid`	Prop	Dữ liệu record đi từ parent/list xuống card; receiving child không sở hữu nguồn dữ liệu.
`show`	State	Giá trị local thay đổi theo click và quyết định Modal hidden/visible.

State transition bắt buộc phải giải thích được

```
View Detail Click
  ↓
onClick handler
  ↓
setShow(true)
  ↓
show = true
  ↓
React renders again
  ↓
Modal visible
```

13. Quyết định Context: dùng hay không dùng

Mặc dù Buổi 04 đã học Context, Buổi 05 không ép Context vào Orchid Gallery. Với cây `OrchidList` → `OrchidCard`, data chỉ đi một direct prop hợp nên Props thường đơn giản và rõ hơn.

Tình huống	Quyết định hợp lý
OrchidList truyền một record trực tiếp cho OrchidCard	Dùng Props.
Navbar và Footer đều cần current user qua nhiều intermediate components	Có thể cân nhắc Context.
Muốn “dùng Context cho đủ bài” dù không có prop drilling	Không nên.

Bảng chứng hợp lệ — Sinh viên có thể ghi “Context Not Needed Yet” và giải thích 2–3 câu. Không dùng một cơ chế khi chưa cần cũng là quyết định kiến trúc tốt.

14. Compose ứng dụng hoàn chỉnh

Bước 15. Ghép các component trong App.jsx

THAO TÁC: Giữ `App` ở vai trò compose application. Import NavBar, OrchidList và Footer (nếu có).

VÌ SAO: App không nên chứa toàn bộ data, card markup, modal detail và mọi logic.

KẾT QUẢ MONG ĐỢI: Ứng dụng hiển thị đúng các phần theo component tree.

KIỂM TRA: Trace `main.jsx` → App → OrchidList → OrchidCard` và đối chiếu với cây đã vẽ.

NEU THẤT BẠI: Nếu blank UI, bắt đầu từ App và import/export path trước khi nhìn styling.

```
import AppNavBar from "../components/AppNavBar";
import OrchidList from "../components/OrchidList";
import Footer from "../components/Footer";

function App() {
  return (
    <>
      <AppNavBar />
      <main>
        <OrchidList />
      </main>
      <Footer />
    </>
  );
}

export default App;
```

15. Human-first Debugging Workflow

Trước khi hỏi AI, phải thu thập Expected, Actual, first terminal error, first browser error, relevant file/code và last change. Chỉ sửa một vấn đề mỗi lần.

```
BUILD/UI PROBLEM
↓
Does npm run dev start?
↓
Does browser reach the URL?
↓
Terminal compile error?
↓
Browser console error?
```

```

↓
Import / path / export?
↓
Data / prop / field?
↓
Event / setter / state?
↓
Asset / styling?

```

Triệu chứng	Nguyên nhân hay gặp	Diagnostic nhỏ nhất	Verification
Compile fail	Syntax/import/path/export	Đọc first terminal error và file:line	Server compile lại.
Blank UI	App/child render hoặc runtime error	Console + trace App → child	UI xuất hiện.
Broken image	Sai filename/case/path/data	Mở trực tiếp `/images/...`	HTTP asset load thành công.
Prop undefined	Tên prop/destructure/field lệch	Trace data → map → prop → child	Field render đúng.
State không đổi	Event/handler/setter/JSX không phụ thuộc state	Log/predict từng stage	UI thay đổi sau click.
Modal không đóng	onHide/close handler/setShow(false)	Trace show → onHide → setter	Modal đóng.
Bootstrap không style	CSS import thiếu	Kiểm tra main.jsx + package	Style xuất hiện.
Key warning	Thiếu/stable key không đúng	Dùng domain id	Warning biến mất.

15.1. Hai lỗi kinh điển cần tự phát hiện

```
// SAI: setter được gọi ngay khi render
onClick={setShow(true)}
```

```
// ĐÚNG
onClick={() => setShow(true)}
```

```
// SAI về kiến trúc
OrchidCard1.jsx
OrchidCard2.jsx
OrchidCard3.jsx
```

```
// ĐÚNG mental model
One OrchidCard component + many data objects
```

16. Active Learning và Bug Injection

16.1. Predict the output

```
{OrchidsData.map((orchid) => (
  <OrchidCard
    key={orchid.id}
    orchid={orchid}
  />
))}
```

Nếu `OrchidsData.length === 8`, có bao nhiêu OrchidCard instances được yêu cầu trong lần render này? → Dự đoán trước khi chạy.

16.2. Trace one record

```
{
  id: "2",
  orchidName: "Mini Yaya",
  category: "Dendrobium"
}
```

Trace: Data module → map variable → `orchid` prop → destructuring → `orchid.orchidName` → Card.Title.

16.3. Bug injection

- Giảng viên đổi `orchidName` thành `orchidname` trong child. Nhóm phải xác định mismatch bằng data trace.
- Giảng viên tạm bỏ Bootstrap CSS import. Sinh viên phân biệt “React vẫn chạy” với “styling dependency bị thiếu”.
- Giảng viên đổi `key={orchid.id}` thành thiếu key để sinh viên đọc warning, không dùng random key để che warning.

17. AI-Augmented Workflow: AI sau evidence

17.1. AI-0 Design Task

Không dùng AI: với yêu cầu “hiển thị 8 Orchid; mỗi item có image, name, category, special status; View Detail hiển thị additional info”, sinh viên phải tự viết Component Tree, Data Source, Props, State, Event và Expected UI.

17.2. Prompt review kiến trúc

Act as a senior React lab reviewer.

Requirement:
[REQUIREMENT]

My component tree:
[TREE]

My data flow:
[DATA FLOW]

My state ownership:
[STATE]

Do NOT generate code.

Review only:

1. component responsibilities;
2. props flow;
3. state ownership;
4. unnecessary Context;
5. duplicated components.

Keep recommendations within SBA301 beginner scope.

17.3. Prompt trace Prop

My Orchid data is:
[DATA]

Parent code:
[CODE]

Child code:
[CODE]

Actual result:

[RESULT]

Do not rewrite files. Trace exactly:
 Data object → map callback → prop → destructuring → field access → JSX.
 Identify the FIRST mismatch.

17.4. Prompt trace State

My Detail button does not work.

Expected: Modal opens.
 State: [STATE CODE]
 Button: [BUTTON CODE]
 Modal: [MODAL CODE]
 Console: [ERROR]

Do not rewrite the component. Trace:
 Click → Event Handler → State Setter → New State → Modal show value.
 Identify the first broken stage.

17.5. Hallucination checks

Phát biểu	Đánh giá đúng
“OrchidsData.js là database vì lưu Orchid lâu dài.”	Sai. Đây là static frontend source data, không phải database.
“useState tự cập nhật backend khi mở Modal.”	Sai. Đây chỉ là client component state; chưa có Spring Boot communication.

Overengineering guard — Trong lab này không tự động thêm Redux, Zustand, React Query, TypeScript, Router, Axios, global Context store hay custom Hook abstraction. Nội dung ngoài phạm vi phải ghi “Learn Later”.

18. Git checkpoint

Bước 16. Commit milestone đã được verify

THAO TÁC: Chỉ commit sau khi app chạy, cards đúng, interaction hoạt động và console sạch relevant errors.

VÌ SAO: Git checkpoint tạo history có thể quay lại và evidence về increment hoàn chỉnh.

KẾT QUẢ MONG ĐỢI: `git log -1` cho thấy commit mới với message có nghĩa.

KIỂM TRA: Chạy `git status`, review files, add, commit, rồi `git log -1`.

NẾU THẤT BẠI: Nếu commit thất bại do config, xử lý Git identity/config; không coi screenshot UI là thay thế Git evidence.

```
git status
git add .
git commit -m "Integrate orchid gallery with props and state"
git log -1
```

19. Evidence Package và Verification Matrix

Screenshot một màn hình đẹp không đủ chứng minh hiểu bài. Evidence phải bao phủ design, runtime, data flow, state và version control.

- ☐ Component Tree đã cập nhật theo implementation thật.
- ☐ Screenshot Orchid Gallery.
- ☐ Screenshot Detail/Modal state (nếu bridge task được thực hiện).
- ☐ Browser Console không có blocking/relevant error.
- ☐ Terminal dev server không có compile error.
- ☐ Git commit hash.
- ☐ Một đoạn giải thích data flow 5–7 câu.

Requirement	Evidence	Result
App starts	Terminal + browser	PASS / FAIL
NavBar	Browser	PASS / FAIL
Orchid list	Browser	PASS / FAIL
Correct count	Data vs UI	PASS / FAIL
Props	Code trace	PASS / FAIL
Images	Browser/Network	PASS / FAIL
map() + key	Code + Console	PASS / FAIL
useState	Code	PASS / FAIL
Button event	Interaction	PASS / FAIL
Detail open/close	Interaction	PASS / FAIL
Console	DevTools	PASS / FAIL
Git commit	git log	PASS / FAIL

19.1. Data Flow Explanation Template

Câu hoàn chỉnh	Sinh viên điền
Orchid data begins in:	_____
The list gets data by:	_____
One Orchid is passed to:	_____
The prop is called:	_____
The detail state belongs to:	_____
Clicking the button calls:	_____
The UI changes because:	_____

20. Rubric tự đánh giá lab

Tiêu chí	0	1	2	3
Functional Correctness	Không chạy	Chạy một phần	Core chạy, còn minor issue	Core + interaction ổn định

Tiêu chí	0	1	2	3
Component Structure	Dồn App	Tách hình thức	Component có trách nhiệm	Tree rõ, không overcomponent
Props/Data Flow	Không hiểu	Prop có nhưng lẫn lộn	Trace được một record	Reuse + map + trace rõ
State/Interaction	Không có	Có state nhưng lỗi	Open hoặc close đúng	Open/close + giải thích state flow
Verification Evidence	Không evidence	Chỉ screenshot	Có runtime evidence	Đủ design/runtime/Git
Student Explanation	Không giải thích	Nhờ cú pháp	Giải thích từng phần	Giải thích ownership + future API bridge

Điểm thi giác không được lẫn át functional correctness, data flow và khả năng giải thích.

21. Từ static data đến full-stack

```

HÔM NAY
OrchidsData.js
↓
React

SAU NÀY
Spring Boot REST API
↓ JSON
React

FULL-STACK
SQL Server
↓
JPA
↓
Spring Boot
↓ REST/JSON
React

```

Mental model quan trọng — Data module hôm nay được dùng để luyện component/data flow. Nó sẽ được thay bằng API data ở các buổi full-stack, không phải database ẩn bên trong React.

22. Chuyển giao sang Group Project

Sau Orchid lab, mỗi nhóm áp dụng đúng pattern vào domain của mình, không copy “Orchid” về mặt nghiệp vụ.

Domain	Data	List	Reusable Item
News	NewsData	NewsList	NewsCard
Hotel	RoomData	RoomList	RoomCard
Course	CourseData	CourseList	CourseCard
Inventory	ProductData	ProductList	ProductCard

22.1. Mini-deliverable bắt buộc của nhóm

- ☐ Ít nhất 4 sample records.
- ☐ Một reusable item component.

- ☐ Props truyền record vào item.
- ☐ Array rendering bằng map().
- ☐ Stable key.
- ☐ Một local state interaction (View Detail / Show-Hide / Favorite / Expand).
- ☐ React-Bootstrap layout.
- ☐ Không backend ở thời điểm này.
- ☐ Có Git commit và giải thích data flow.

23. Human Verification Gate (không dùng AI)

11. Lab 01 chính thức nhằm đạt các mục tiêu chính nào?
12. Vite đóng vai trò gì?
13. Tại sao phải cài React-Bootstrap?
14. Tại sao Bootstrap CSS phải import riêng?
15. Lab 01 đặt thư mục ảnh ở đâu?
16. OrchidList chịu trách nhiệm gì?
17. OrchidCard chịu trách nhiệm gì?
18. Tại sao chỉ nên có một OrchidCard component?
19. Props là gì?
20. Tại sao Props read-only?
21. Một Orchid đi từ data module đến OrchidCard như thế nào?
22. `map()` làm gì?
23. `map()` có phải React Hook không?
24. Tại sao collection cần stable `key`?
25. State là gì?
26. `useState(false)` biểu diễn điều gì?
27. Khi click View Detail, chuỗi event → state → UI diễn ra thế nào?
28. Giá trị nào điều khiển Modal visibility?
29. `useState` có lưu dữ liệu vào database không?
30. `OrchidsData.js` có phải database không?
31. Tại sao Context không bắt buộc trong app này?
32. Evidence nào chứng minh app hoạt động?
33. Lỗi runtime/browser xem ở đâu?
34. Tại sao cần Git checkpoint?
35. Tương lai thành phần nào sẽ thay static Orchid data?

Điều kiện đạt — Một output chạy được nhưng sinh viên không giải thích được Props/State/data ownership chưa được xem là đạt năng lực tích hợp React.

24. Phiếu phân tích kết quả

Nhóm	Nội dung cần ghi	Kết quả
APPLICATION	Development URL / number of Orchids / number rendered	_____
ARCHITECTURE	Root / List / Item / Detail component	_____
PROPS	Prop / source / receiver	_____
STATE	State / initial / trigger / updated	_____

Nhóm	Nội dung cần ghi	Kết quả
EVIDENCE	Terminal / Console / Images / Detail	PASS / FAIL
GIT	Commit message / hash	_____

25. Exit Ticket

Q1. Điền flow

```

OrchidsData
  ↓
  _____
  ↓
OrchidCard
  ↓
Browser

```

Q2. Vì sao `<OrchidCard orchid={orchid} />` tốt hơn tạo 8 component khác nhau?

Trả lời bằng khái niệm reusability + one component + different data, không chỉ nói “code ngắn hơn”.

Q3. Điền state flow

```

View Detail Click
  ↓
  _____
  ↓
show = true
  ↓
  _____

```

Q4. Vì sao `OrchidsData.js` không phải database?

Q5. Nội dung nào trong buổi là bridge từ Lab 02?

26. Reflection và Home Practice

26.1. Reflection

- The first time my application worked end-to-end was when...
- The component structure is...
- Data starts at...
- Props move data to...
- State controls...
- The hardest bug was...
- The evidence that revealed it was...
- AI suggested..., and I verified/rejected it by...
- If I rebuilt the lab, I would...

26.2. Home Practice — Group Project

☐ 5+ records.

- ☐ Reusable Item component.
- ☐ Props.
- ☐ map().
- ☐ stable key.
- ☐ one local state interaction.
- ☐ React-Bootstrap layout.
- ☐ no blocking errors.
- ☐ meaningful Git commit.
- ☐ prepare to explain code without AI.

26.3. Optional challenges

Challenge	Yêu cầu	Điểm học tập
A — Filter	Show Special Orchids Only bằng local state; không mutate original data.	Derived/displayed subset.
B — Selected Item State	Parent giữ selectedOrchid + showModal; một Modal dùng chung.	State ownership / lifting awareness.
C — Empty State	Khi filter có 0 item, hiển thị “No orchids found.”	UX state awareness.

27. Glossary nền tảng

Thuật ngữ	Định nghĩa ngắn	Ví dụ lab	Vì sao quan trọng
Integration Lab	Bài thực hành tích hợp nhiều kiến thức đã học	Buổi 05	Chứng minh khả năng ghép kiến thức
Component Tree	Cây biểu diễn hierarchy component	App → OrchidList → OrchidCard	Thiết kế trước code
Data Module	Module chứa static source data	OrchidsData.js	Tách data khỏi UI
Static Data	Dữ liệu nằm trong source frontend	Orchid array	Khác database/API
Props	Input từ parent đến child	orchid prop	Tái sử dụng component
Read-only	Receiver không sửa trực tiếp input	Không mutate orchid prop	One-way data flow
State	Dữ liệu component nhớ và thay đổi	show	Tương tác UI
useState	Hook tạo local state	[show, setShow]	State trong function component
Event Handler	Hàm xử lý tương tác	onClick	Nối user action với state
Re-render	React render lại khi state/inputs thay đổi	Modal xuất hiện	Declarative UI
Collection	Tập nhiều record	OrchidsData	Nguồn list UI
map()	JavaScript Array method	Record → OrchidCard	Render collection
key	Stable identifier cho item collection	orchid.id	React tracking list items
React-Bootstrap	Bootstrap components cho React	Card, Modal, Grid	Reusable UI
Card	UI container cho một item	One orchid	Repeated item UI
Modal	Overlay detail UI	Orchid detail	Bridge state interaction
Container/Row/Col	Grid layout primitives	Responsive orchid list	Responsive layout
Asset	Tài nguyên tĩnh	image file	UI content
public	Thư mục static assets của Vite project	public/images	Browser-accessible assets

Thuật ngữ	Định nghĩa ngắn	Ví dụ lab	Vì sao quan trọng
Debugging Evidence	Dữ liệu chứng minh lỗi	terminal + console + code	Fix có căn cứ
Git Commit	Checkpoint source code	integrate orchid gallery...	Traceable development
Regression	Feature cũ hỏng sau thay đổi	Card mất style sau refactor	Preview professional habit

28. Timeline 135 phút

Phút	Hoạt động	Output
00–10	Pre-lab check + success criteria	Readiness
10–20	Baseline project / deps / assets	Working baseline
20–30	Component tree + data-flow design	Design evidence
30–45	Data module + one OrchidCard with Props	One record works
45–60	map() + key + full list	Collection rendered
60–73	React-Bootstrap Grid/Card + image check	Responsive list
73–88	useState + event	State trace
88–102	Detail/Modal bridge	Open/close interaction
102–112	Debugging + injected bugs	Evidence-based fix
112–120	AI-assisted review/debug	Reviewed solution
120–127	Transfer to group project	Mini feature design
127–131	Git checkpoint	Commit hash
131–135	Human verification + exit ticket	Learning evidence

Nếu lớp yếu — Ưu tiên: Working project → Component tree → Props → map() → State → Detail. Không hy sinh hiểu data flow chỉ để hoàn thiện Modal đẹp.

29. Cầu nối sang Session 06 — React Router & Navigation

Sau Buổi 05, sinh viên có component architecture, Props, State và một interactive UI. Tuy nhiên ứng dụng vẫn chủ yếu là một rendered screen. Câu hỏi mới là: “Nếu có Home, Orchids, About và Detail thì làm thế nào chuyển giữa các views mà không tải lại toàn bộ website?”

SESSION 06

```

SPA
↓
Router
↓
Routes
↓
Route
↓
Link / NavLink
↓
URL Parameters

```

↓
Navigation

Không học trước — Không đưa Router/Axios/REST/backend/database vào Buổi 05. Mỗi cơ chế mới phải xuất hiện khi sinh viên đã đủ mental model để giải thích nó.

PHỤ LỤC A — Đáp án gợi ý Exit Ticket

Q1: ``OrchidList / map()`` là mắt xích giữa `OrchidsData` và `OrchidCard`.

Q2: Một component có thể tái sử dụng với nhiều object khác nhau qua Props; tránh nhân bản implementation và giữ trách nhiệm nhất quán.

Q3: ``event handler/state setter`` → ``show=true`` → ``Modal/detail rendered``.

Q4: ``OrchidsData.js`` là static source data trong frontend; không cung cấp persistence/database engine/query/transaction và không sống như một DB service.

Q5: Structured Orchid detail data và Modal/detail interaction là bridge từ Lab 02.

PHỤ LỤC B — Checklist cuối buổi

- ☐ Tôi mở đúng project và ``npm run dev`` chạy.
- ☐ Tôi hiểu component tree của app.
- ☐ Tôi biết nơi static data bắt đầu.
- ☐ Tôi dùng một `OrchidCard` tái sử dụng.
- ☐ Tôi trace được một record qua Props.
- ☐ Tôi dùng `map()` và stable key.
- ☐ Tôi phân biệt `map()` và React Hook.
- ☐ Tôi giải thích được `show / setShow / false`.
- ☐ Tôi dự đoán state trước khi click.
- ☐ Tôi mở/đóng detail thành công hoặc giải thích core state interaction.
- ☐ Tôi biết Context chưa cần thiết nếu data flow chỉ một hop.
- ☐ Tôi biết xem terminal và browser console.
- ☐ Tôi có evidence và Git commit.
- ☐ Tôi không dùng AI thay cho component/data-flow reasoning.
- ☐ Tôi sẵn sàng học Router ở Session 06.